



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



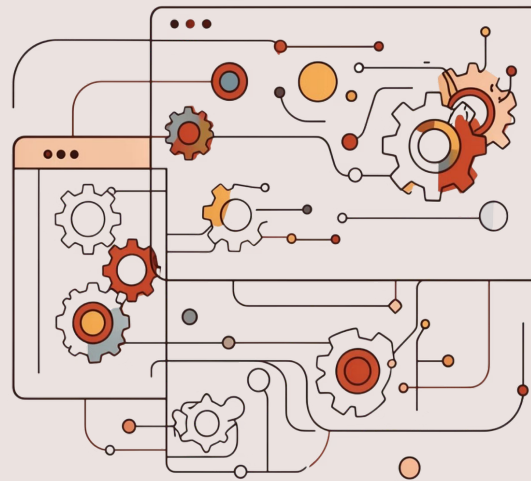
MARCO TULIO VALENTE





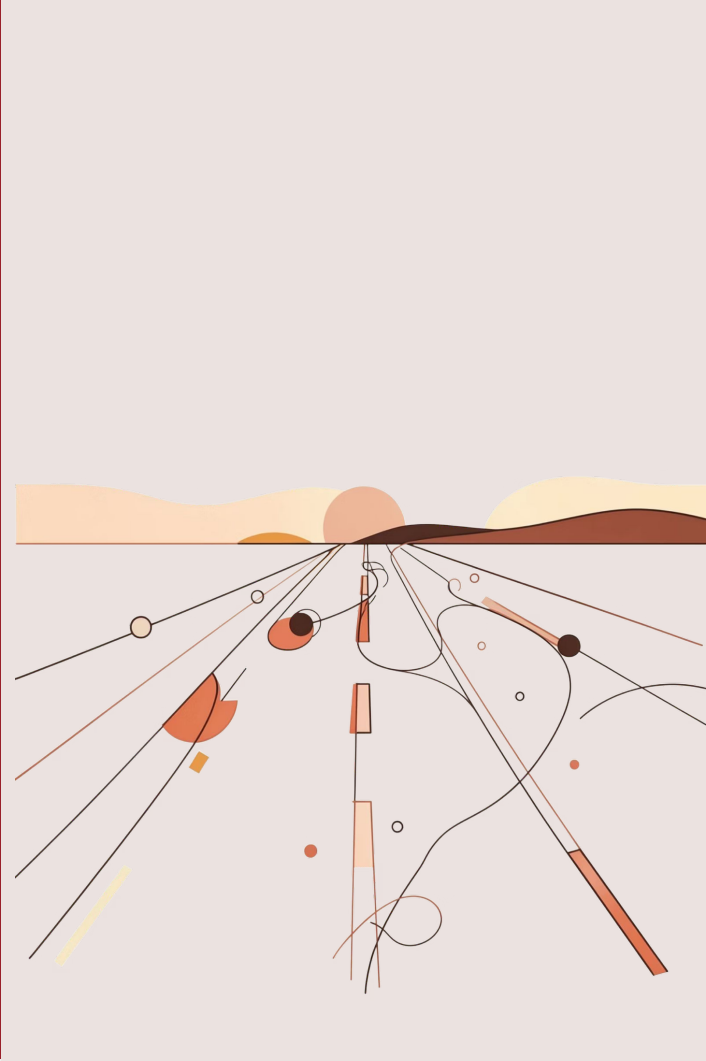
Manutenção, Evolução e Dívida Técnica

O Ciclo de Vida do Software, compreendendo tipos de manutenção e reconhecendo a dívida técnica acumulada nos projetos.





O Software Nunca Está "Pronto"



A Ilusão da Finalização



Software é um **organismo vivo**, em constante transformação. As necessidades do negócio evoluem, as tecnologias avançam e os usuários mudam seus comportamentos e expectativas.

⚠ Um software "pronto" hoje pode ser obsoleto amanhã. A estabilidade é uma ilusão, a evolução é a realidade.



A Necessidade da Manutenção

Manter o software funcionando e relevante não é opcional — é **essencial para a sobrevivência do produto**. Sem manutenção contínua, o software se degrada progressivamente e perde valor competitivo.



Funcionalidade

Garantir que o sistema opere sem falhas no dia a dia.



Relevância

Acompanhar as mudanças do mercado e do usuário.



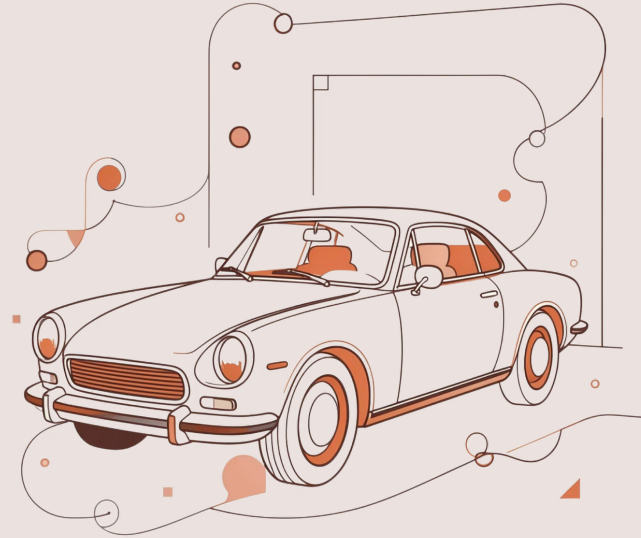
Segurança

Proteger contra vulnerabilidades e ameaças emergentes.



Manutenção Constante = Longevidade

Assim como um carro clássico bem conservado mantém seu valor e desempenho ao longo das décadas, um software bem mantido permanece eficiente, seguro e relevante, independentemente da sua idade.





Os Quatro Pilares da Manutenção de Software



Manutenção Corretiva

O Bombeiro Digital

Resposta imediata a falhas e incidentes que afetam a operação do sistema em produção.

O que envolve?

- Correção de bugs e falhas reportadas
- Resposta a incidentes críticos inesperados
- Restauração da funcionalidade comprometida

 Exemplo: Um cliente reporta que o botão de "salvar" não funciona, a equipe age imediatamente para corrigir.



Manutenção Adaptativa

A Adaptação ao Mundo

Ajustes necessários para que o software continue operando em ambientes tecnológicos em constante mudança.

O que envolve?

- Compatibilidade com novos sistemas operacionais
- Atualização de navegadores e bibliotecas externas
- Integração com novas APIs e plataformas

 Exemplo: Atualizar o software para rodar corretamente em uma nova versão do Windows ou no macOS mais recente.



Manutenção Evolutiva



Crescimento e Novas Funcionalidades

Expansão do sistema com novos recursos que atendem às demandas crescentes do negócio e dos usuários.

O que envolve?

- Adição de novos módulos e funcionalidades
- Melhorias de performance e usabilidade
- Expansão para novos casos de uso



Exemplo: Implementar um novo módulo de relatórios gerenciais no sistema existente.



Manutenção Preventiva

Antecipando Problemas

Ações proativas para melhorar a qualidade interna do código antes que os problemas se manifestem.

O que envolve?

- Refatoração de código para melhorar a qualidade
- Otimização de performance antes que vire gargalo
- Atualização de dependências para evitar vulnerabilidades

✓ Exemplo: Reescrever uma seção complexa de código para torná-la mais legível, testável e eficiente.



Os Quatro Tipos em Perspectiva



Corretiva

Corrige o que quebrou



Adaptativa

Ajusta ao ambiente



Evolutiva

Adiciona valor



Preventiva

Antecipa problemas

Cada tipo de manutenção desempenha um papel único no ciclo de vida do software. A combinação equilibrada dessas práticas é o que garante a **evolução contínua e o valor agregado** ao longo do tempo.

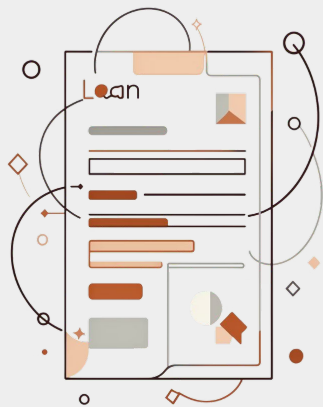




A Sombra da Dívida Técnica



O Que é Dívida Técnica?



Metáfora criada por **Ward Cunningham**, um dos autores do Manifesto Ágil, a dívida técnica representa o **custo futuro das escolhas de desenvolvimento feitas hoje**.

É como contrair um empréstimo: você obtém algo rápido agora, mas paga juros — com atrasos, bugs e complexidade crescente — depois.



Por Que Empresas Acumulam Dívida Técnica?



Pressão por Prazos

Lançar rápido no mercado é prioridade, atalhos viram padrão.



Falta de Experiência

Equipes inexperientes podem não reconhecer boas práticas de desenvolvimento.



Decisões de Arquitetura

Escolhas técnicas que se mostram inadequadas apenas a longo prazo.



Falta de Documentação

Dificulta a compreensão e manutenção do código por novos membros.



Atalhos de Hoje, Problemas de Amanhã

Cada atalho tomado sob pressão é uma conta que será cobrada no futuro, geralmente com juros. A dívida técnica não desaparece; ela cresce silenciosamente até se tornar um obstáculo crítico para o projeto.



Sinais de Dívida Técnica Elevada

Reconhecer os sintomas é o primeiro passo para agir. Fique atento a estes alertas:



Atrasos Recorrentes

Entregas sempre atrasam sem motivo aparente



Código Complexo

Pequenas mudanças consomem muito tempo da equipe



Alta Frequência de Bugs

O sistema apresenta instabilidade frequente em produção



Desmotivação da Equipe

Desenvolvedores frustrados ao trabalhar no código



Tipos de Dívida Técnica

Segundo **Martin Fowler**, a dívida técnica pode ser classificada em duas categorias fundamentais:

Dívida Deliberada

Escolha **consciente** de uma solução mais rápida, com plano de refatoração futura. A equipe sabe o que está fazendo e por quê.

- ✅ Exemplo: "Vamos usar essa abordagem simples agora e refatorar na próxima sprint."

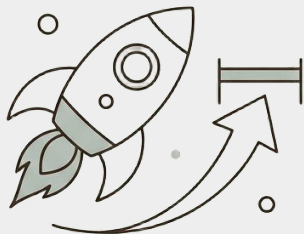
Dívida Acidental

Resultado de **má prática ou falta de conhecimento** — a equipe não percebeu que estava criando um problema futuro.

- ❌ Exemplo: Código duplicado espalhado pelo sistema porque o desenvolvedor não conhecia padrões de reuso.



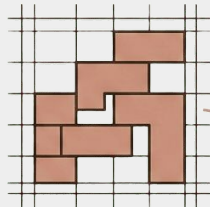
O Equilíbrio Frágil



VELOCIDADE



Lançar rápido.
Entregar features.
Conquistar mercado.



QUALIDADE



Código limpo. Testes.
Documentação.
Manutenibilidade.

A tensão entre **velocidade de entrega** e **qualidade técnica** é uma das mais antigas da engenharia de software.

O segredo não está em escolher um lado, mas em **gerenciar conscientemente as concessões**, sabendo quando e quanto de dívida técnica é aceitável assumir.





A Dívida Técnica no Nosso Projeto



Reflexão: Onde Estamos?

Antes de agir, precisamos **enxergar com clareza**. Esta análise nos convida a olhar honestamente para o estado atual do nosso projeto.

1 Identificar as áreas com sinais evidentes de dívida técnica

Quais módulos geram mais bugs? Quais levam mais tempo para alterar?

2 Compreender as decisões que nos trouxeram aqui

Foram pressões de prazo? Falta de conhecimento? Mudanças de requisitos?

3 Mapear o impacto real no dia a dia da equipe

Atrasos, retrabalho, desmotivação e instabilidade em produção.



Identificando Dívidas Específicas

Exemplos concretos de dívida técnica identificados no nosso projeto:

1

Módulo de Relatórios

Código legado acumulado ao longo de anos, difícil de adaptar para novas exigências de negócio. Qualquer mudança requer análise extensa.

2

Integração com API Externa

Dependência de biblioteca desatualizada com falhas de segurança conhecidas. A atualização foi postergada por risco de quebra de funcionalidades.

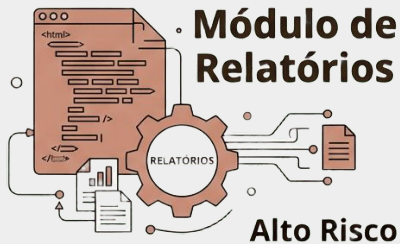
3

Falta de Testes Automatizados

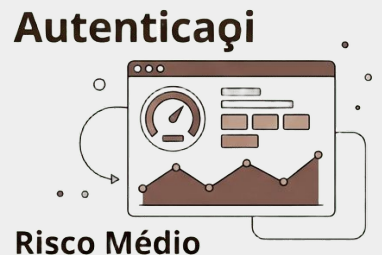
A ausência de uma suite de testes aumenta significativamente o tempo de identificação e correção de bugs em produção.



Mapeando o Risco



Visualizar a distribuição da dívida técnica no mapa de arquitetura ajuda a **priorizar onde intervir primeiro**.



Áreas de **alto risco** exigem atenção imediata; áreas de baixo risco podem ser tratadas de forma gradual e planejada.



⚠ A dívida técnica não distribuída de forma uniforme, ela tende a se concentrar nos módulos mais antigos e mais modificados.





Gerenciando e Pagando a Dívida Técnica



A Importância de Priorizar

**Alto impacto
+ Fácil**



Pagar agora

**Alto impacto
+ Difícil**



Planejar

**Baixo impacto
+ Fácil**



**Aproveitar
oportunidade**

**Baixo impacto
+ Difícil**



**Aceitar
por enquanto**

Nem toda dívida técnica precisa ser paga imediatamente. A chave está em **priorizar estrategicamente** com base no impacto real no negócio.

Como decidir?

- Qual o impacto no negócio se não resolvermos?
- Qual a complexidade de resolver agora?
- Ferramentas de análise estática de código podem ajudar a mensurar

☐ Ferramentas como SonarQube, CodeClimate e ESLint ajudam a quantificar e monitorar a dívida técnica de forma objetiva.

Estratégias para Gerenciamento



Alocar Tempo para Refatoração

Dedicar uma porcentagem fixa do sprint (ex: 20%) exclusivamente para pagar dívidas técnicas identificadas.



Refatorar ao Implementar Novos Recursos

Sempre que tocar em um módulo para adicionar funcionalidade, aproveitar para melhorar o código ao redor, a regra do escoteiro.



Manter Documentação Atualizada

Documentar decisões técnicas e arquiteturais evita que a mesma dívida seja contraída novamente no futuro.





Planejar Juntos é Pagar Mais Rápido

O gerenciamento da dívida técnica é um esforço coletivo. Quando a equipe inteira participa do planejamento das refatorações, as soluções são mais robustas e o comprometimento é maior.



O Papel da Prevenção

A melhor dívida é aquela que nunca é contraída. Práticas preventivas reduzem drasticamente o acúmulo futuro:



Padrões de Codificação

Manter consistência e legibilidade em todo o código do projeto.



Testes Automatizados

Garantir qualidade e agilidade nas refatorações futuras.



Revisões de Código

Compartilhar conhecimento e identificar problemas antes do merge.



Dependências Conscientes

Evitar inchaço com frameworks e bibliotecas desnecessárias.





Prevenir é Melhor que Remediar

Investir em qualidade desde o início é sempre mais barato do que corrigir problemas acumulados. Um conjunto sólido de práticas preventivas é o escudo mais eficaz contra a dívida técnica.



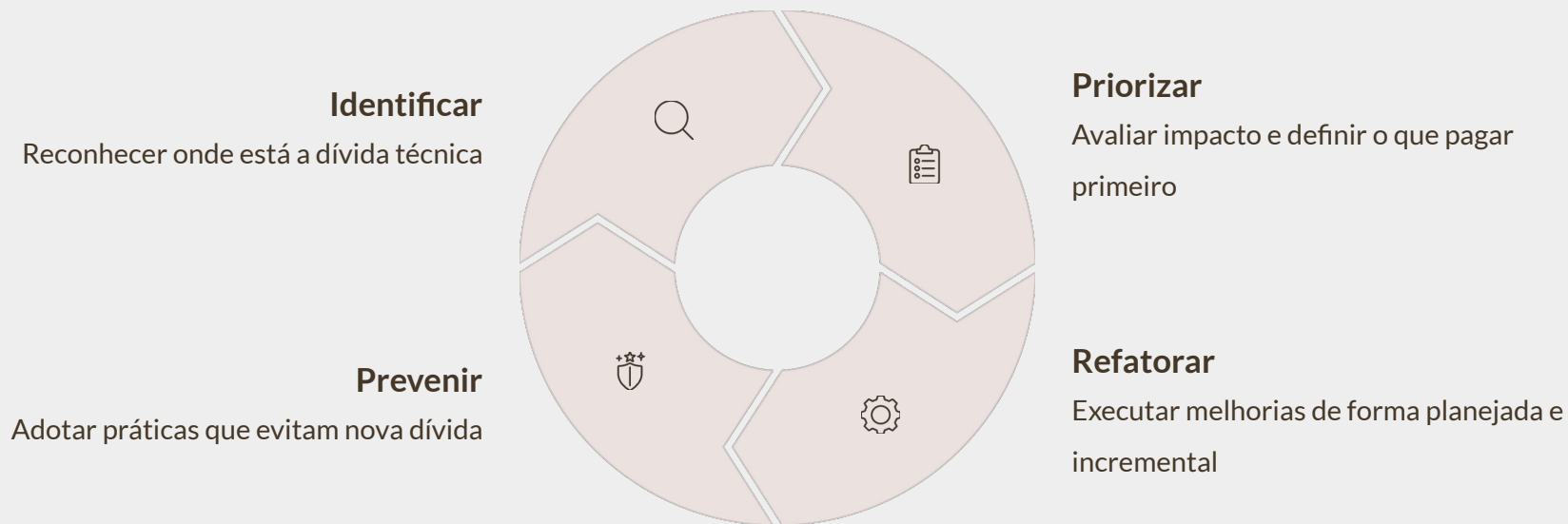


O Futuro é Evolução Constante



Conclusão: Um Ciclo de Melhoria Contínua

Software é um processo contínuo, não um produto acabado. A dívida técnica é um **desafio gerenciável**, não um obstáculo intransponível.



✓ Ao entender, identificar e gerenciar ativamente a dívida técnica, garantimos a **saúde e o sucesso a longo prazo** do nosso projeto.



Vamos debater?



Vamos debater?

Refactoring

introdução sobre o tema

questões

referências

análises

ativar modo anônimo

o modo anônimo permite responder as questões temporariamente de forma anônima. ao desabilitar, as pessoas que responderam passam a ser identificadas.

introdução sobre o tema

O refactoring, ou refatoração de código, consiste em um conjunto de técnicas disciplinadas orientadas para a melhoria interna da estrutura de um sistema de software sem alterar seu comportamento externo observável. Este processo tem se consolidado como prática fundamental dentro da Engenharia de Software, sobretudo nas abordagens ágeis e no contexto de desenvolvimento contínuo, onde a manutenção da qualidade e a sustentabilidade do código ao longo do tempo são essenciais. A importância do refactoring está ancorada na necessidade de lidar com a complexidade crescente dos sistemas, promovendo um ambiente onde novas funcionalidades podem ser implementadas de maneira mais ágil, confiável e segura.

questão 1

Como você saberia, sem testes automatizados, que um refactoring não quebrou nada? É possível ter essa certeza?

3 respostas · 3 comentários

questão 2

De que forma a identificação de code smells pode se trans-

introdução sobre o tema

O refactoring, ou refatoração de código, consiste em um conjunto de técnicas disciplinadas orientadas para a melhoria interna da estrutura de um sistema de software sem alterar seu comportamento externo observável. Este processo tem se consolidado como prática fundamental dentro da Engenharia de Software, sobretudo nas abordagens ágeis e no contexto de desenvolvimento contínuo, onde a manutenção da qualidade e a sustentabilidade do código ao longo do tempo são essenciais. A importância do refactoring está ancorada na necessidade de lidar com a complexidade crescente dos sistemas, promovendo um ambiente onde novas funcionalidades podem ser implementadas de maneira mais ágil, confiável e segura.

questão 1

Como você saberia, sem testes automatizados, que um refactoring não quebrou nada? É possível ter essa certeza?

3 respostas · 3 comentários

questão 2

De que forma a identificação de code smells pode se trans-

Manutenção, evolução e dívida técnica

introdução sobre o tema

questões

referências

análises

ativar modo anônimo

o modo anônimo permite responder as questões temporariamente de forma anônima. ao desabilitar, as pessoas que responderam passam a ser identificadas.

introdução sobre o tema

A manutenção de software é um campo fundamental na Engenharia de Software, permeando todo o ciclo de vida de sistemas e aplicações. Diferentemente de artefatos físicos tradicionais, o software é essencialmente dinâmico e evolutivo por natureza, sendo constantemente impactado por contextos tecnológicos, mudanças de negócios e necessidades dos usuários. A ideia de que um software pode ser considerado 'pronto' é um equívoco comum, pois, na prática, todo produto digital demanda acompanhamento contínuo para garantir seu funcionamento adequado, relevância e alinhamento com os objetivos da organização.

questão 1

O software nunca está 'pronto', mas há um momento em que ele está 'bom o suficiente'. Como uma equipe decide onde está esse limite? Quem deveria decidir?

3 respostas · 3 comentários

questão 2

introdução sobre o tema

A manutenção de software é um campo fundamental na Engenharia de Software, permeando todo o ciclo de vida de sistemas e aplicações. Diferentemente de artefatos físicos tradicionais, o software é essencialmente dinâmico e evolutivo por natureza, sendo constantemente impactado por contextos tecnológicos, mudanças de negócios e necessidades dos usuários. A ideia de que um software pode ser considerado 'pronto' é um equívoco comum, pois, na prática, todo produto digital demanda acompanhamento contínuo para garantir seu funcionamento adequado, relevância e alinhamento com os objetivos da organização.

questão 1

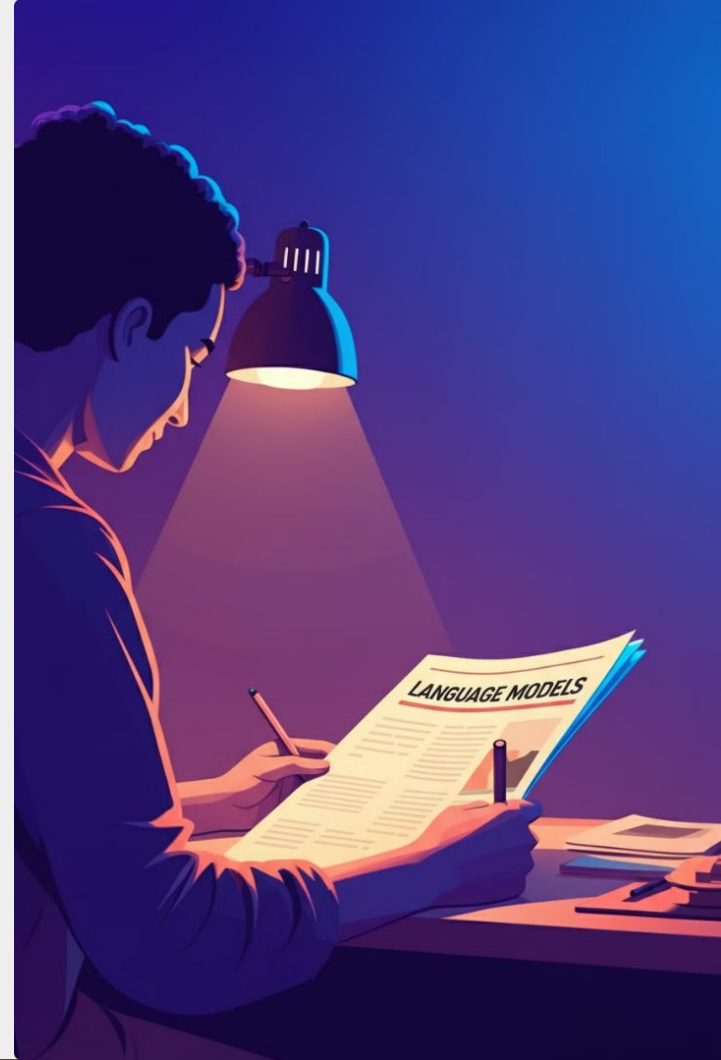
O software nunca está 'pronto', mas há um momento em que ele está 'bom o suficiente'. Como uma equipe decide onde está esse limite? Quem deveria decidir?

3 respostas · 3 comentários

questão 2

Próximos passos

- Eng. Software em Dimensões
 - Cap. 19 (seções 19.1–19.4):
Manutenção, refactoring,
dívida técnica, documentação
- Eng. Software Moderna
 - Cap. 9 – RefactoringCatálogo,
code smells, prática



contacts

vcg@cin.ufpe.br
@vinicius3w
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Manutenção, Evolução e Dívida
Técnica